

Devoir maison 2 — Fonctions, répétition, simulation et performances

UE INF-STAR — Année 2026-2027

Oskar Laverny

Invalid Date

Table des matières

Informations pratiques	1
Contexte	1
Exercice 1 — Écrire des fonctions réutilisables — 4 points	1
Exercice 2 — Appliquer ses fonctions aux données — 6 points	2
Exercice 3 — Boucle, vectorisation et vérification — 3 points	2
Exercice 4 — Simulation — 3 points	3
Exercice 5 — Benchmarking et reproductibilité — 4 points	3
Présentation du rendu	4

Informations pratiques

Ce devoir est **individuel**. Il mobilise principalement les parties 05 à 09 du cours : boucles, fonctions, vectorisation, `lapply()`, `sapply()`, simulation, benchmarking et reproductibilité.

- **Données** : `data/crocodile_dataset.csv`.
- **Rendu attendu** : `NOM_Prenom_DM2.qmd` et son PDF rendu avec Quarto.

Le document doit être reproductible : une personne disposant du `.qmd` et du fichier CSV doit pouvoir reconstruire le PDF depuis une session propre. Utilisez uniquement des chemins relatifs.

Ajoutez une annexe exécutant `sessionInfo()` pour décrire l'environnement utilisé. L'utilisation de `renv` n'est pas exigée pour ce rendu.

Contexte

Le fichier fourni contient des observations de crocodiliens avec notamment leur longueur, leur poids, leur habitat et leur statut de conservation. Dans ce devoir, il sert avant tout de support à des exercices de programmation ; il ne vous est pas demandé d'en tirer des conclusions biologiques générales.

Exercice 1 — Écrire des fonctions réutilisables — 4 points

1. Écrivez une fonction `indice_masse()` prenant en arguments une longueur en mètres et un poids en kilogrammes et renvoyant :

$$\frac{poids}{longueur^2}$$

Cet indice est utilisé ici uniquement comme **exercice de programmation** ; il ne doit pas être interprété comme un indicateur biologique validé.

2. La fonction doit accepter des vecteurs de longueur et de poids.
3. Écrivez une fonction `resume_num()` qui prend un vecteur numérique et renvoie un vecteur nommé contenant au minimum :

- `n` ;
- `moyenne` ;
- `ecart_type` ;
- `minimum` ;
- `maximum`.

4. Testez vos deux fonctions sur quelques valeurs simples avant de les utiliser sur le jeu de données.

Pour les résumés, `n` désigne le nombre de valeurs observées, hors valeurs manquantes. Calculez les autres statistiques sur ces mêmes valeurs.

Exercice 2 — Appliquer ses fonctions aux données — 6 points

À partir du jeu de données :

1. sélectionnez les colonnes quantitatives `Observed Length (m)` et `Observed Weight (kg)` ;
2. utilisez `lapply()` et votre fonction `resume_num()` pour produire un résumé des deux colonnes ;
3. utilisez `sapply()` pour obtenir leur moyenne ;
4. sélectionnez avec une condition logique les observations d'un habitat de votre choix et calculez leur longueur moyenne ;
5. appliquez directement votre fonction vectorisée `indice_masse()` aux deux colonnes pour obtenir un indice par observation ;
6. expliquez en quelques phrases les différences de **forme de sortie** entre `lapply()` et `sapply()`.

Ignorez les valeurs manquantes dans les résumés. Pour la sélection d'un habitat, excluez explicitement les valeurs manquantes du critère.

Exercice 3 — Boucle, vectorisation et vérification — 3 points

Calculez la longueur au carré pour chaque observation de deux façons :

1. avec une boucle `for` et un vecteur résultat préalloué ;
2. avec une expression vectorisée.

Vérifiez que les deux résultats sont identiques avec `all.equal()`.

Expliquez pourquoi la version vectorisée est plus courte sans en conclure automatiquement qu'elle sera toujours plus rapide.

Exercice 4 — Simulation — 3 points

On souhaite étudier la variabilité de la moyenne de longueur obtenue sur des échantillons aléatoires du jeu de données.

1. Fixez la graine aléatoire à 27. Retirez les longueurs manquantes avant les tirages ; utilisez cette même population de valeurs observées pour les deux tailles d'échantillon.
2. À l'aide de `replicate()`, répétez **2 000 fois** l'expérience suivante :
 - tirer 50 observations avec remise ;
 - calculer la longueur moyenne de l'échantillon.
3. Représentez la distribution des 2 000 moyennes simulées.
4. Recommencez avec des échantillons de taille 200.
5. Comparez l'écart-type des moyennes simulées dans les deux situations et interprétez en une ou deux phrases.

Exercice 5 — Benchmarking et reproductibilité — 4 points

On souhaite calculer la moyenne de chaque colonne d'une matrice numérique construite à partir des variables quantitatives du jeu de données.

Comparez au moins les trois approches suivantes :

1. une boucle `for` avec préallocation ;
2. `apply()` sur les colonnes du tableau numérique ;
3. une fonction spécialisée adaptée, par exemple `colMeans()` lorsque cela est possible.

Utilisez `bench::mark()` avec une vérification de l'égalité des résultats.

Utilisez la même règle pour les valeurs manquantes dans les trois méthodes. Les sorties doivent avoir les mêmes noms : conservez les noms de colonnes dans le vecteur préalloué de la boucle. Vérifiez avec `all.equal()` avant le benchmark et gardez la vérification de `bench::mark()` activée.

Dans votre rapport :

- affichez le benchmark ;
- identifiez la méthode la plus rapide sur votre machine ;
- comparez approximativement les allocations mémoire ;
- expliquez pourquoi il faut d'abord vérifier que les méthodes donnent le même résultat avant de comparer leurs performances ;
- précisez en une phrase pourquoi un benchmark réalisé sur votre ordinateur n'est pas une constante universelle.

Présentation du rendu

Votre document Quarto doit contenir :

- un titre et votre nom ;
- une courte introduction ;
- une section par exercice ;
- le code nécessaire et les résultats utiles ;
- les graphiques produits ;
- des commentaires courts répondant aux questions d'interprétation.
- une annexe contenant la sortie de `sessionInfo()`.

On n'attend pas un long rapport : la priorité est donnée à un code clair, correct et reproductible.